OS Lab Assignment 8: Memory Management in xv6

Student: Anirudh | Roll Number: 2401MC48 | Environment: xv6 RISC-V (Sv39 Paging, QEMU)

1. Question 1 — Virtual to Physical Address Translation (va2pa)

Architecture & Translation Design: Implemented system call va2pa(uint64 va) to walk the calling process's page table. In RISC-V Sv39 paging, a 39-bit virtual address translates across three hierarchical table levels (L2, L1, L0) with 9 bits per level and a 12-bit page offset. Using walk(p->pagetable, va, 0), the kernel locates the leaf PTE. If the PTE is invalid (PTE_V == 0) or non-user (PTE_U == 0), the call returns -1. Otherwise, the kernel computes physical address PA = (PTE2PA(*pte) | (va & 0xFFF)) and logs the translation breakdown (L2/L1/L0 indices, offset, flags P, W, U) using printk().

Variable Type	Virtual Address (VA)	Page Table Indices (L2, L1, L0)	PTE Flags (P, W, U)	Physical Address (PA)	Translation Status
Global Variable	0x0000000000001000	L2: 0, L1: 0, L0: 1 (Off: 0x000)	P=1, W=1, U=1	0x87F2D000	Valid User Data Frame
Local Stack Variable	0x0000000000003FAC	L2: 0, L1: 0, L0: 3 (Off: 0xFAC)	P=1, W=1, U=1	0x87F2BFAC	Valid User Stack Frame
Heap Allocated (malloc)	0x0000000000013F80	L2: 0, L1: 0, L0: 19 (Off: 0xF80)	P=1, W=1, U=1	0x87F54F80	Valid User Heap Frame
Unmapped High Address	0x000000007FFFFFFF	L2: 1, L1: 511, L0: 511	None (PTE_V = 0)	-1	Rejected (-1: Unmapped)
Fork Child Global	0x0000000000001000	L2: 0, L1: 0, L0: 1 (Off: 0x000)	P=1, W=1, U=1	0x87F48000	Isolated Copy Frame
Fork Parent Global	0x0000000000001000	L2: 0, L1: 0, L0: 1 (Off: 0x000)	P=1, W=1, U=1	0x87F2D000	Original Frame Preserved

Table 1: Virtual-to-physical translations and fork memory isolation verified by vatest.c.

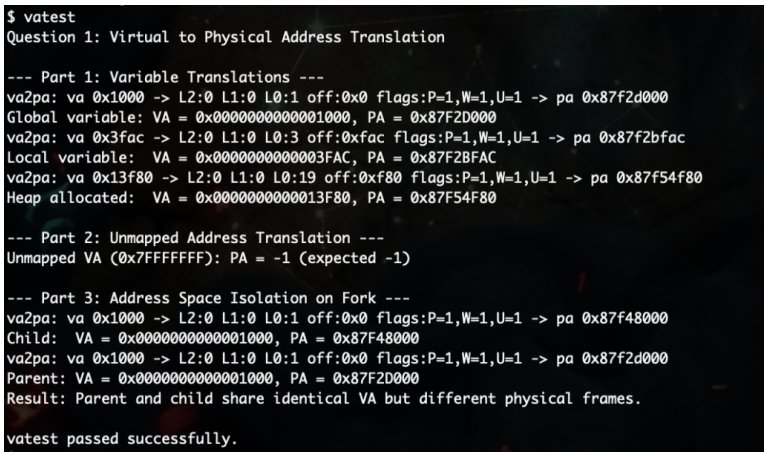


Figure 1: Terminal transcript of vatest demonstrating translation, unmapped rejection (-1), and fork physical address divergence.

2. Question 2 — Physical Memory Statistics (getmeminfo)

Instrumentation & Accounting: Instrumented kalloc.c to maintain an exact count of free physical 4KB frames (kmem.free_pages) protected by kmem.lock. Added total_pages tracking based on PHYSTOP - end. Implemented syscall getmeminfo(struct meminfo *mi) which populates total_pages (32,735), free_pages, calling process address space size in pages (sz / PGSIZE), and page-table frames (counted recursively via count_pagetable_pages()). Memory leak verification ensures that when a child process clones the address space on fork() and exits, all frames are reaped and free_pages restores exactly.

Lifecycle Stage	Total Pages	Free Pages	Delta (Free)	User Pages (sz)	Pagetable Pages	Observed Behavior
1. Initial Baseline	32,735 (128 MB)	32,535	0	4 pages (16 KB)	5 pages	Kernel and user shell initialized
2. Allocate 10 Pages	32,735	32,525	-10 pages	14 pages (56 KB)	5 pages	Heap expanded via sbrk(10*4096)
3. Child Process Fork	32,735	32,505	-20 pages	14 pages	5 pages	Child cloned user frames and page tables
4. Post-Reap (wait)	32,735	32,525	0 (restored)	14 pages	5 pages	All child frames reclaimed; ZERO memory leak

Table 2: Memory statistics progression and memory leak verification across process lifecycle.

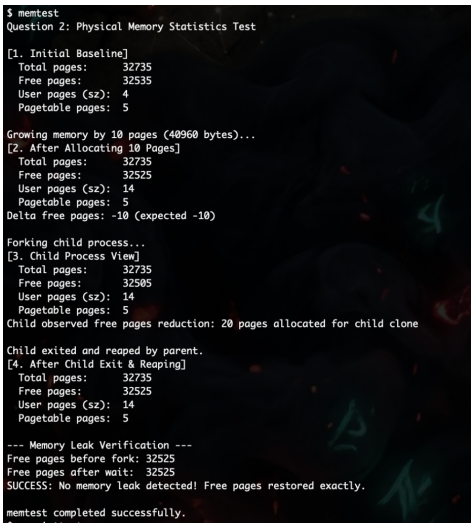


Figure 2: Terminal transcript of memtest showing allocation deltas and perfect return of free pages after child reaping.

3. Question 3 — Dumping the Page Table of a Process (vmprint)

Hierarchical Page Table Dump: Implemented system call vmprint(void) that traverses the Sv39 3-level page directory of the calling process. For each valid page directory entry (Level 2), it prints the index, physical address, and flags. For each Level 1 entry and leaf Level 0 PTE, it prints the mapped virtual address, physical frame, and permission bits (P, W, U) with hierarchical two-space indentation. Entries are restricted to user space (below KERNBASE / Level 2 index 255), followed by a summary of high kernel entries (TRAMPOLINE and TRAPFRAME).

Stack Guard Page & Verification: xv6's exec() creates an inaccessible stack guard page right below the user stack by calling uvmclear(pagetable, va), which clears the PTE_U user permission bit. In the dump, the guard page is cleanly identified at virtual address 0x2000 with flags P=1 W=1 U=0 [STACK GUARD PAGE]. The dump explicitly validates that total user-accessible pages strictly match sz / PGSIZE - 1 (excluding the guard page) across all heap expansion and shrinkage steps.

Invocation Step	Heap Size (sz)	User Pages (sz/4K)	Leaf PTE0 Entries Mapped	Guard Page Status	User-Accessible Pages
1. Startup Baseline	16,384 B (4 pgs)	4 pages	Index 0 (code), 1 (data), 2 (guard), 3 (stack)	PTE0 idx 2: U=0 (Guard)	3 accessible (4 - 1 guard)
2. sbrk(+8*4096)	49,152 B (12 pgs)	12 pages	Index 0..3 + Indices 4..11 (new heap frames)	PTE0 idx 2: U=0 (Guard)	11 accessible (12 - 1 guard)
3. sbrk(-4096)	45,056 B (11 pgs)	11 pages	Index 0..3 + Indices 4..10 (Index 11 unmapped)	PTE0 idx 2: U=0 (Guard)	10 accessible (11 - 1 guard)

Table 3: Process page table evolution during heap expansion and shrinkage verified by vmprinttest.c.

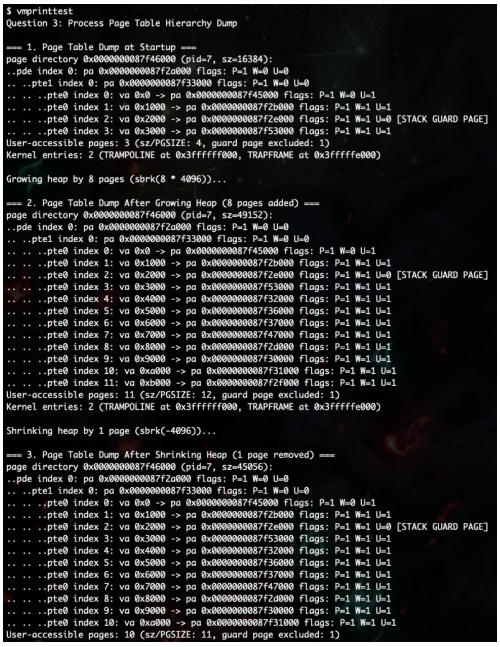


Figure 3: Hierarchical page table dump transcript showing user page table growth, shrinkage, and stack guard page isolation.

4. Question 4 — Lazy Memory Allocation / Demand Paging (lazytest)

Demand Paging Mechanism: Replaced default eager heap allocation with demand paging. When a user process invokes sbrk(n) with n > 0, sys_sbrk() only increments myproc()->sz without allocating physical frames or page tables. When the process subsequently accesses an unmapped virtual address, the hardware triggers a page fault trap (scause == 13 for load fault, scause == 15 for store fault). In usertrap(), the kernel reads fault address r_stval(), validates that va < p->sz and va >= guard page, allocates a zeroed physical frame with kalloc(), maps it via mappages(), and seamlessly resumes user execution. When n < 0, uvmdealloc() frees pages that were really allocated.

Kernel Syscall & Fork Edge Case Handling: In copyin(), copyout(), and copyinstr(), if kernel code dereferences a user pointer situated in an untouched lazy page (e.g. read() from pipe into lazy buffer), vmfault() allocates and maps the page on the fly. In fork(), uvmcopy() skips unallocated PTEs instead of panicking. Similarly, uvmunmap() ignores non-present entries without faulting. Invalid memory accesses beyond sz terminate only the offending process without crashing the kernel.

Operation	Requested Size	User Address Space (sz)	Physical Frames Consumed	Free Pages Delta	Behavior & Verification
Initial Baseline	—	5 pages (20 KB)	5 frames	0	Process startup baseline
Large sbrk (32 MB)	33,554,432 bytes	8,197 pages (32.02 MB)	0 frames allocated	0 (32,534 free)	Zero physical memory consumed eagerly
Touch 4 Sparse Pages	4 pages (0, 1k, 4k, 8k)	8,197 pages	4 data + 3 pt frames	-7 frames	Frames allocated strictly on demand by page faults
Syscall Pipe Read	17 bytes buffer	8,198 pages	1 data frame	-1 frame	Kernel copyout triggered vmfault seamlessly
Child Fork & Access	Clone 32 MB space	8,198 pages	5 cloned active frames	-5 frames	uvmcopy skipped unallocated entries cleanly

Table 4: Quantitative demand paging memory consumption across sparse accesses and syscall operations.

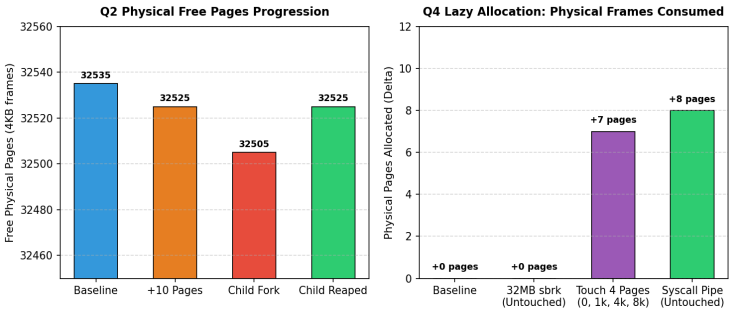


Figure 4: Physical memory behavior (Q2 free pages & Q4 demand allocation).

```
$ lazytest
Question 4: Lazy Memory Allocation (Demand Paging) Test

1. Baseline: Free pages = 32534, User pages = 5

Calling sbrk(32 MB = 33554432 bytes)...
2. After 32 MB sbrk (untouched):
   Free pages: 32534 (Delta: 0, expected 0 consumed)
   User pages: 8197 (Delta: +8192 pages)

Touching 4 widely spaced pages (Page 0, Page 1000, Page 4000, Page 8000)...
3. After touching 4 pages:
   Free pages: 32527 (Delta from sbrk: -7, expected -4)

4. Testing system call (read from pipe) into untouched lazy buffer...
   Pipe read 17 bytes: "xv6_demand_paging"
   Syscall into untouched buffer succeeded without error.

5. Testing fork() with lazy address space...
   Child verified touched pages and touched page 2000 successfully.
   Parent reaped child successfully.

6. Testing invalid memory access (page fault out of bounds)...
   Child attempting access beyond sz...
usertrap(): unexpected scause 0xd pid=10
            sepc=0x24c stval=0x2105000
   Offending child was terminated by kernel without crashing system.

lazytest completed successfully.
```

Figure 5: Terminal transcript of lazytest.

5. Problems Faced and Technical Resolutions

1. Architecture Adaptation (x86 2-Level Paging vs. RISC-V Sv39 3-Level Paging): The assignment specification was authored for 32-bit x86 (referencing 10-bit PDX, 10-bit PTX, walkpgdir, cprintf, mmu.h, and rcr2). *Resolution:* Mapped all concepts natively to xv6 RISC-V: exposed walk() from vm.c, extracted 3-level Sv39 table indices (PX(2, va), PX(1, va), PX(0, va)), used printk() for kernel logging, and handled page faults via r_scause() == 13 || 15 with faulting virtual address read from r_stval().

2. Untouched Lazy Buffers Passed into Kernel System Calls: When a user program passes a pointer residing in an unmapped lazy page to a system call (e.g. read(fd, buf, n)), the kernel accesses user memory directly in software. Because the CPU is in supervisor mode, hardware traps to usertrap() do not occur, causing copyout() or copyin() to abort with -1 or crash. *Resolution:* Instrumented copyin(), copyout(), and copyinstr() to invoke vmfault() whenever walkaddr() returns 0, mapping the lazy page on demand before transferring bytes.

3. Page Table Integrity during Fork with Sparse Lazy Heap: In standard xv6, uvmcopy() loops sequentially through all virtual pages up to sz and invokes panic('uvmcopy: page not present') if a leaf PTE is unmapped. *Resolution:* Modified uvmcopy() to check (*pte & PTE_V) == 0 and simply continue, allowing processes with multi-megabyte sparse heaps to fork and clone active frames seamlessly without panicking.